



Theme Park · Petting Zoo — Helper Functions — English Worksheet (Advanced)

Topic: Petting Zoo — Helper Functions · **Course:** Theme Park Creator · **Level:** Advanced · **Time:** about 50 minutes

Your park has stalls and amenities. Now it needs a **petting zoo** — a fence around it, six cages inside, an animal in every cage, and somewhere to eat and drink.

Every cage is the same job. So you write that job **once**, then **call** it for every animal.

These are the tools you use this lesson:

```
corner1 = pos(1, 0, 1)
corner2 = pos(10, 0, 10)
blocks.fill(OAK_FENCE, corner1, corner2, FillOperation.REPLACE)
mobs.spawn(CHICKEN, pos(6, 0, 6))

for i in range(10):
    mobs.spawn(CHICKEN, pos(6, 0, 6))
```

`corner1` and `corner2` are the **two corners** of the box you fill. `FillOperation.REPLACE` fills the **whole solid** box. `AIR` is a block like any other — filling with `AIR` **deletes** what was there. `mobs.spawn(animal, pos(x, y, z))` puts **one** animal in **one** spot. `for i in range(10):` runs the line under it **10 times**. `OAK_FENCE` = the fence. `HAY_BLOCK` = the food. `WATER` = the drink.

These are the tools. **Turning the numbers into parameters is your job.**

1 · Solid, Then Carve

There is no "hollow box" block. So you build a box in **two moves**:

move 1 – fill it solid

```
  1  2  3  4  5  6
6  ## ## ## ## ## ##
5  ## ## ## ## ## ##
4  ## ## ## ## ## ##
3  ## ## ## ## ## ##
2  ## ## ## ## ## ##
1  ## ## ## ## ## ##
```

-->

move 2 – carve the middle out

```
  1  2  3  4  5  6
6  ## ## ## ## ## ##
5  ## . . . . ##
4  ## . . . . ##
3  ## . . . . ##
2  ## . . . . ##
1  ## ## ## ## ## ##
```

fill OAK_FENCE

(1, 1) to (6, 6)

fill AIR, one block in

(2, 2) to (5, 5)

Move 2 is one block inside move 1 on every side. The edge it leaves behind is your fence.

2 · Functions 🧩

What a helper function is

	Regular function	Helper function
Who calls it?	You do. You type it and run it.	Another function does.
How big is the job?	The whole thing.	One small job.
In your zoo	<code>build_zoo</code>	<code>fence</code> · <code>feed_area</code>
Written differently?	No — same <code>def</code> .	No — same <code>def</code> .

⚠️ There is **no special keyword** for a helper. A helper is just a function that **other functions use**. The word describes its **job**, not its code.

How to tell them apart: look at who calls it. If you type it yourself, it's the regular one. If only other functions type it, it's a helper.

Functions inside functions 🙋

A function can **call** another function. When it does, it **stops and waits** — the second function runs all the way to the end, then the first one carries on from where it stopped.

Follow the arrows. `-->` means *go into*. `<--` means *finished, come back*.

```

YOU TYPE: build_zoo(2, 2, 12, 12)
|
| line 1 --> fence(OAK_FENCE, 2, 2, 12, 12)           [level 1]
|             |-- fills the outer box
|             <-- back to build_zoo
|
| line 2 --> build_cage(CHICKEN, 1, 1, 10, 10)         [level 1]
|             |
|             |--> fence(OAK_FENCE, 1, 1, 10, 10)     [level 2]
|             |             |-- the SAME helper, smaller
|             |             <-- back to build_cage
|             |
|             |-- for i in range(10): spawn CHICKEN
|             |
|             |--> feed_area(1, 1)                     [level 2]
|             |             |-- hay, then sunken water
|             |             <-- back to build_cage
|             <-- back to build_zoo
|
| line 3 --> build_cage(COW, 13, 1, 22, 10)
v             <-- ... and so on, six times

```

The rule: nothing is skipped. `build_zoo` cannot get to its next line until the function it called has completely finished.

Work it out step by step:

Step 1. You type `build_zoo(2, 2, 12, 12)`. One call. Everything else follows from it.

Step 2. `build_zoo` calls `fence`. `build_zoo` **pauses** — one function waiting.

Step 3. `fence` ends. `build_zoo` **resumes** at line 2 and calls `build_cage`.

Step 4. `build_cage` calls `fence`. Now **two** are paused: `build_zoo` and `build_cage`. This is the chain at its deepest.

Step 5. `fence` ends → control returns to `build_cage`, **not** to `build_zoo`. A function always returns to **whoever called it**.

Step 6. `build_cage` spawns, calls `feed_area`, waits again, gets control back, and ends.

Step 7. Only *now* does `build_zoo` reach line 3.

Count it up: one `build_zoo`, seven `fence` runs, six `build_cage` runs, six `feed_area` runs. You wrote each function **once**.

How to write a helper

Four steps, every time:

Step 1 — Find the job that repeats. The zoo fence is a box. Every cage is a box. One job, seven runs.

Step 2 — Write the job once. Give it a name that says the job.

Step 3 — Put the changing parts in the brackets. Those become **parameters**.

Step 4 — Call it, with different arguments each time.

Here is the **plan** for `build_cage`. The code is yours to write — this is only the order of the jobs:

```
def build_cage(animal, x, z, x2, z2):  
    # 1. fill a solid box of fence, corner to corner  
    # 2. carve the middle back to AIR, one block in  
    # 3. put the animal inside  
    # 4. add the feeding area
```

3 · Predict 🧠

Read each one. Write what happens **and the reason**.

```
blocks.fill(OAK_FENCE, pos(1, 0, 1), pos(10, 0, 10),  
            FillOperation.REPLACE)  
blocks.fill(AIR, pos(2, 0, 2), pos(9, 0, 9), FillOperation.REPLACE)
```

What is left, how thick is it, and why is move 2 inset by exactly one?


```
def fence(fence, x, z, x2, z2):  
    corner1 = pos(x - 5, 0, z - 5)  
    corner2 = pos(x2 + 25, 0, z2 + 25)  
    ...  
  
fence(OAK_FENCE, 2, 2, 12, 12)
```

Which corners does this box actually get? They are not the ones passed in — work them out, and explain what the `- 5` and `+ 25` are doing.


```
for i in range(10):  
    mobs.spawn(animal, pos(6, 0, 6))
```

How many animals, and where? `i` is never used inside the loop — does that matter? Why?

4 · Spot the Bug 🐛

Bug A — This should leave a fence ring with a space inside.

```
blocks.fill(OAK_FENCE, pos(1, 0, 1), pos(10, 0, 10),  
            FillOperation.REPLACE)  
blocks.fill(AIR, pos(1, 0, 1), pos(10, 0, 10),  
            FillOperation.REPLACE)
```

What is left, and why?

Write the fixed lines. Explain why your fix works.

Bug B — Read this one carefully. It runs without an error.

```
def build_zoo(x, z, x2, z2):  
    fence(OAK_FENCE, x, z, x2, z2)  
    build_cage(CHICKEN, 1, 1, 10, 10)  
    build_cage(COW, 13, 1, 22, 10)  
    build_cage(PIG, 25, 1, 34, 10)  
  
build_zoo(2, 2, 12, 12)  
build_zoo(40, 2, 50, 12)
```

Work it out step by step:

Step 1. Write down what `x`, `z`, `x2`, `z2` are on the **second** call.

Step 2. Which line uses those four values? Which lines do **not**?

Step 3. So on the second call — what moves, and what stays exactly where it was?

Step 4. How would you change `build_zoo` so the whole zoo moves together — fence, cages and animals? Do not write the whole function. Write the idea, and one line to show it.

5 · Build the Petting Zoo 🐑

In your homework world, write **one** helper and three functions that call down into it:

```
def fence(fence, x, z, x2, z2):
def build_cage(animal, x, z, x2, z2):
def feed_area(x, z):
def build_zoo(x, z, x2, z2):
```

`fence` builds one box: solid fill, then carve the middle to `AIR`.

`build_cage` calls `fence`, puts its animal inside, then calls `feed_area`.

`feed_area` fills `HAY_BLOCK` for food and `WATER` for drink. Sink the water one block down.

`build_zoo` calls `fence` once for the outer fence, then calls `build_cage` **six** times with **six** different animals.

Only `fence` may contain a fence `blocks.fill`. Every box in the zoo comes from that one helper.

Build the chain from the bottom up. Do not write `build_zoo` first.

Step 1. Write `fence`. Test it alone. It is the bottom of the chain — nothing above it can work until it does.

Step 2. Write `feed_area`. Test it alone.

Step 3. Write `build_cage`. Its first line calls `fence`. No fence `blocks.fill` in here.

Step 4. Test `build_cage` with **one** call before you write six.

Step 5. Write `build_zoo` last. It is the only function you call yourself.

Step 6. Run it. Walk the whole zoo. Every cage: fence, animal, food, water.

Animals: `CHICKEN` · `COW` · `PIG` · `SHEEP` · `RABBIT` · `LLAMA`

Cage corners that fit — front row and back row:

```
z 13-22  [ SHEEP ]  [ RABBIT ]  [ LLAMA ]
z  1-10  [ CHICKEN]  [  COW  ]  [  PIG  ]
          x 1-10      x 13-22    x 25-34
```

Write your helper and your three functions:

Trace one animal down your chain. Start at `build_zoo`, end at the block that gets placed. Which values are passed at each step, and which function hands control back to which?

6 · Show Your Work 📷 🎥

Record **one video** — one take, no stopping (a phone is fine). Show these in order:

1 · Your code. Scroll through it. Say what each part does.

2 · Your build. Point the camera. Name the parts.

3 · Your chain. Point at each function and say who calls who, and who is a helper.

Say these out loud, filling in the blanks:

Today I built _____.

I built it using this code: _____.

In this code I used _____.

The hardest part was _____.

That part was hard because _____.

The most fun part was _____.

Something new I learned was _____.

Submit 

Send this worksheet + your video to teacher on KakaoTalk.

Write your lines here, then say them in your video.